P1021R3
Mike Spertus, Symantec
mike_spertus@symantec.com
Timur Doumler
papers@timur.audio
Richard Smith
richardsmith@google.com
2018-11-26
Audience: Core Working Group

Filling holes in Class Template Argument Deduction

This paper proposes filling several gaps in Class Template Argument Deduction.

Document revision history

R0, 2018-05-07: Initial version
R1, 2018-10-07: Refocused paper on filling holes in CTAD
R2, 2018-11-26: Following EWG guidance, removed proposal for CTAD from partial template argument lists
R3, 2019-01-21: Add wording and change target to Core Working Group

Rationale

As one of us (Timur) has noted when giving public presentations on using class template argument deduction, there are a significant number of cases where it cannot be used. This always deflates the positive feelings from the rest of the talk because it is accurately regarded as artificially inconsistent. In particular, listeners are invariably surprised that it does not work with aggregate templates, type aliases, and inherited constructors. We will show in this paper that these limitations can be safely removed. Note that some of these items were intentionally deferred from C++17 with the intent of adding them in C++20.

Class Template Argument Deduction for aggregates

We propose that Class Template Argument Deduction works for aggregate initialization as one shouldn't have to choose between having aggregates and deduction. This is well illustrated by the following example:

C++17	Proposed

`template<class T>`
`struct Point { T x; T y; };`
 	
`// Aggregate: Cannot deduce`
`Point<double> p{3.0, 4.0};`
`Point<double> p2{.x = 3.0, .y = 4.0};`
	`template<class T>`
	`struct Point { T x; T y; };`
	
	`// Proposed: Aggregates deduce`
	`Point p{3.0, 4.0};`
	`Point p2{.x = 3.0, .y = 4.0};`

For the code on the right hand side to work in C++17, the user would have to write an explicit deduction guide. We believe that this is unnecessary and error-prone boilerplate and that the necessary deduction guide should be implicitly synthesized by the compiler from the arguments of a braced or designated initializer during aggregate initialization.

Algorithm

In the current Working Draft, an aggregate class is defined as a class with no user-declared or inherited constructors, no private or protected non-static data members, no virtual functions, and no virtual, private or protected base classes. While we would like to generate an aggregate deduction guide for class templates that comply with these rules, we first need to consider the case where there is a dependent base class that may have virtual functions, which would violate the rules for aggregates. Fortunately, that case does not cause a problem because any deduction guides that require one or more arguments will result in a compile-time error after instantiation, and non-aggregate classes without user-declared or inherited constructors generate a zero-argument deduction guide anyway. Based on the above, we can safely generate an aggregate deduction guide for class templates that comply with aggregate rules.

When P0960R0 was discussed in Rapperswil, it was voted that in order to allow aggregate initialization from a parenthesized list of arguments, aggregate initialization should proceed as if there was a synthesized constructor. We can use the same approach to also synthesize the required additional deduction guide during class template argument deduction as follows:

1. Given a primary class template `C`, determine whether it satisfies all the conditions for an aggregate class ([dcl.init.aggr]/1.1 - 1.4).
2. If yes, let `T_1`, ..., `T_n` denote the types of the `N` elements ([dcl.init.aggr]/2) of the aggregate (which may or may not depend on its template arguments).
3. Form a hypothetical constructor `C(T_1, ..., T_N)`.
4. For every constructor argument of type `T_i`, if all types `T_i ... T_n` are default-constructible, add a default argument value zero-initializing the argument as if by `T_i = {}`.
5. For the set of functions and function templates formed for [over.match.class.deduct], add an additional function template derived from this hypothetical constructor as described in [over.match.class.deduct]/1.

There is a slight complication resulting from subaggregates, and the fact that nested braces can be omitted when instantiating them:

```
struct Foo { int x, y; };
struct Bar { Foo f; int z; };
Bar bar{1, 2};   // Initializes bar.f.x and bar.f.y to 1; zero-initializes bar.z
```

In this case, we have two initializers, but they do not map to the two elements of the aggregate type `Bar`, instead initializing the sub-elements of the first subaggregate element of type `Foo`.

For complicated nested aggregates, there are potentially many different combinations of valid mappings of initializers to subaggregate elements. It would be unpractical to create hypothetical constructors for all of those combinations. Additionally, whether or not an aggregate type has subaggregate elements may depend on the template arguments:

```
template <typename T>
struct Bar { T f; int z; };  // T might be a subaggregate!
```

This information is not available during class template argument deduction, because for this we first need to deduce `T`. We therefore propose simply to avoid all of these problems by prohibiting the omission of nested braces when performing class template argument deduction.

Class Template Argument Deduction for alias templates

While Class Template Argument Deduction makes type inferencing easier when constructing classes, it doesn't work for type aliases, penalizing the use of type aliases and creating unnecessary inconsistency.We propose allowing Class Template Argument Deduction for type aliases as in the following example.

vector	pmr::vector (C++17)	pmr::vector (proposed)
`vector v = {1, 2, 3};` `vector v2(cont.begin(), cont.end());`	`pmr::vector<int> v = {1, 2, 3};` `pmr::vector<decltype(cont)::value_type> v2(cont.begin(), cont.end());`	`pmr::vector v = {1, 2, 3};` `pmr::vector v2(cont.begin(), cont.e`

`pmr::vector` also serves to illustrate another interesting case. While one might be tempted to write

```
pmr::vector pv({1, 2, 3}, mem_res); // Ill-formed (C++17 and proposed)
```

this example is ill-formed by the normal rules of template argument deduction because `pmr::memory_resource` fails to deduce `pmr::polymorphic_allocator<int>` in the second argument.

While this is to be expected, one suspects that had class template argument deduction for alias templates been around when `pmr::vector` was being designed, the committee would have considered allowing such an inference as safe and useful in this context. If that was desired, it could easily have been achieved by indicating that the `pmr::allocator` template parameter should be considered non-deducible:

```
namespace pmr {
  template<typename T>
  using vector = std::vector<T, type_identity_t<pmr::polymorphic_allocator<T>>>; // See P0887R1 for type_identity_t
}
pmr::vector pv({1, 2, 3}, mem_res); // OK with this definition
```

Finally, in the spirit of alias templates being simply an alias for the type, we do not propose allowing the programmer to write explicit deduction guides specifically for an alias template.

Algorithm

For deriving deduction guides for the alias templates from guides in the class, we use the following approach (for which we are very grateful for the invaluable assistance of Richard Smith):

1. Deduce template parameters for the deduction guide by deducing the right hand side of the deduction guide from the alias template. We do not require that this deduces all the template parameters as nondeducible contexts may of course occur in general
2. Substitute any deductions made back into the deduction guides. Since the previous step may not have deduced all template parameters of the deduction guide, the new guide may have template parameters from both the type alias and the original deduction guide.
3. Derive the corresponding deduction guide for the alias template by deducing the alias from the result type. Note that a constraint may be necessary as whether and how to deduce the alias from the result type may depend on the actual argument types.
4. The guide generated from the copy deduction guide should be given the precedence associated with copy deduction guides during overload resolution

Let us illustrate this process with an example. Consider the following example:

```
template<class T> using P = pair<int, T>;
```

Naively using the deduction guides from pair is not ideal because they cannot necessarily deduce objects of type `P` even from arguments that should obviously work, like `P({}, 0)`. However, let us apply the above procedure. The relevant deduction guide is

```
template<class A, class B> pair(A, B) -> pair<A, B>
```

Deducing `(A, B)` from `(int, T)` yield `int` for `A` and `T` for `B`. Now substitute back into the deduction guide to get a new deduction guide

```
template<class T> pair(int, T) -> pair<int, T>;
```

Deducing the template arguments for the alias template from this gives us the following deduction guide for the alias template

```
template<class T> P(int, T) -> P<T>;
```

A repository of additional expository materials and worked out examples used in the refinement of this algorithm is maintained online.

Deducing from inherited constructors

In C++17, deduction guides (implicit and explicit) are not inherited when constructors are inherited. According to the C++ Core Guidelines C.52, you should “use inheriting constructors to import constructors into a derived class that does not need further explicit initialization”. As the creator of such a thin wrapper has not asked in any way for the derived class to behave differently under construction, our experience is that users are surprised that construction behavior changes:

```
template <typename T> struct CallObserver requires Invocable<T> {
  CallObserver(T &&) : t(std::forward<T>(t)) {}
  virtual void observeCall() { t(); }
  T t;
};

template <typename T> struct CallLogger : public CallObserver<T> {
  using CallObserver<T>::CallObserver;
  virtual void observeCall() override { cout << "calling"; t();  }
};
```

C++17	Proposed

`CallObserver observer([]() { /* ... */ }); // OK`
`CallLogger</*????*/> logger([]() { /* ... */ });`
	`CallObserver observer([]() { /* ... */ }); // OK`
	`CallLogger logger([]() { /* ... */ }); // Now OK`

Note that inheriting the constructors of a base class must include inheriting all the deduction guides, not just the implicit ones. As a number of standard library writers use explicit guides to behave “as-if” their classes were defined as in the standard, such internal implementation details details would become visible if only the internal guides were inherited. We of course use the same algorithm for determining deduction guides for the base class template as described above for alias templates.

Wording

At the end over.match.class.deduct/1, add an addition bullet

- If `C` is defined and satisfies the conditions for an aggregate class ([dcl.init.aggr]/1.1 - 1.4), not considering dependent base classes, let $e_1, \ldots, e_n$ be the elements of `C` ([dcl.init.aggr]/2), not considering dependent base classes, and let $T_1, \ldots, T_n$ be their respective declared types. If `n > 0`, an additional function template, called the aggregate deduction candidate, is derived as above from a hypothetical constructor `C(`T_1`, ..., `T_n`)`. Additionally, for each integer $0 \le$ `i` $<$ `n`, if the types $T_i, \ldots, T_n$ are all default-constructible, the ith parameter of this function template shall have a default argument of the form `{}`.

Add a new subsubsection after over.match.class.deduct titled “over.match.alias.deduct”

When resolving a placeholder for a deduced class type (9.1.7.5) where the *template-name* names an alias template `A` representing a class template specialization of a class `C` with template arguments x_j, a set of functions and function templates are formed as follows. For each function or function template `f` formed for `C` by the process in 11.3.1.8, form a function or function template according to the following procedure:

- Form a hypothetical function or function template `g` with the following properties
  - The template parameters, if any, are the template parameters of `f` (including default template arguments).
  - The argument is an rvalue reference to the result type of `f`
  - The return type is void
- Perform template argument deduction (12.9.2) on `g(C<`x_j`...>{})` with the exception that deduction does not fail if not all template parameters of `g` are deduced.
- Form the desired function or function template as follows
  - The parameters and return type are constructed by the substituting into the parameters (including default arguments) and return type of `f` the deductions of all template parameters that were successfully deduced in the deduction of `g`
  - The template parameters consist of those template parameters of `f` or `g` that appear in the parameters and return type constructed in the previous step
  - The constraints temp.constr.decl (12.4.2) are the conjunction of the constraints of `f` with a constraint that the template parameters of `A` are deducible (12.2.9.5) from the return type
  - If `f` is the copy deduction candidate (11.3.1.8) or was generated from a *deduction-guide* (11.3.1.8) or was generated from a constructor or *deduction guide* with an `explicit-specifier`, then so is the function or function template being formed here

Initialization and overload resolution are performed as described in 9.3 and 11.3.1.3, 11.3.1.4, or 11.3.1.7 (as appropriate for the type of initialization performed) for an object of a hypothetical class type, where the selected functions and function templates are considered to be the constructors of that class type for the purpose of forming an overload set, and the initializer is provided by the context in which class template argument deduction was performed. As an exception, the first phase in 11.3.1.7 (considering initializer-list constructors) is omitted if the initializer list consists of a single expression of type *cv* `U`, where `U` is a specialization of `C` or a class derived from a specialization of `C`. If the function or function template was generated from a constructor or *deduction-guide* that had an *explicit-specifier*, each such notional constructor is considered to have that same *explicit-specifier*. All such notional constructors are considered to be public members of the hypothetical class type. [*Note:* The requires clause ensures that a specialization of `A` can be deduced from the return type. — *end note*]
[*Example:*

```
template <class T, class U> struct C {
  C(T, U);
};
template<class T, class U>
C(T, U) -> C<T, std::type_identity_t<U>>;

template<class V>
using A = C<V *, V *>;

// Possible exposition only implementation of the above procedure
// f is derived (11.3.1.8) from the deduction-guide of C
template<class T, class U> auto f(T, U) -> C<T, std::type_identity_t<U>>;
template<class T, class U> void g(C<T, std::type_identity_t<U>>&&);
// Template argument deduction of g(C<V *, V *>{}) deduces T as V *

//The following concept ensures a specialization of A is deduced
template<T> void t(A<T>&&);
template<T> concept deduces_A = requires { t(std::declval<T>()); };

// Candidate is obtained by transforming f as described by the above procedure
template<class V, class U>
auto f(V *, U) -> C<V, std::type_identity_t<U>>
  requires deduces_A<C<V, std::type_identity_t<U>>>;
// End exposition only
int i{};
double d{};
A a1(&i, &i); // A<int>
A a2(i, i); // ill-formed - fails to match parameter list
A a3(&i, &d); // ill-formed - cannot deduce alias template from C<int *, double *>
```

— end example]

Insert a bullet after over.match.best/1.9 as follows

- `F1` is generated from class template argument deduction (over.match.class.deduct) for a class `D`, `F2` is generated from class template argument deduction for a base class `B` of `D`, and for all arguments the corresponding parameters of `F1` and `F2` have the same type, or if not that,